

LEVEL 5 PROJECT REPORT

Containerized Java/Tomcat Application CI/CD

using Jenkins, Maven, Docker, Amazon ECR and AWS EC2

Technology Stack: GitHub • Jenkins • Maven • Docker • Amazon ECR • Amazon EC2 • Tomcat

AWS Region: Asia Pacific (Mumbai) — ap-south-1

Technology / Service	Role in Project
GitHub	Stores Java application, pom.xml, Dockerfile and Jenkinsfile
Jenkins	Orchestrates checkout, build, image creation, registry push and deployment
Apache Maven	Compiles/tests/packages the application and generates the WAR
Java 17	Build/runtime environment shown in the implementation
Docker	Builds and runs the application container
Amazon ECR	Stores the java-webapp Docker image
Amazon EC2	Hosts Jenkins and the Docker application environment
Apache Tomcat 10	Runs the packaged Java web application
AWS CLI	Used for AWS/ECR operations from the terminal
GitHub Webhook	Connects GitHub push events to Jenkins

Prepared by Manimaran

Implementation Images: 25 screenshots + 1 system architecture diagram

Table of Contents

1. Project Overview
2. Problem Statement
3. Objectives
4. Technologies and Tools
5. Prerequisites
6. System Architecture
7. Complete CI/CD Workflow
8. Project Components
9. Implementation Procedure
10. Error Handling and Resolution
11. Testing and Validation
12. Security Considerations
13. Advantages
14. Limitations
15. Future Enhancements
16. Conclusion
17. Implementation Evidence Summary

1. Project Overview

This project implements an end-to-end Continuous Integration and Continuous Deployment (CI/CD) workflow for a Java web application packaged as a WAR file and deployed inside a Docker container running Apache Tomcat on an Amazon EC2 instance. The source code is maintained in GitHub. Jenkins retrieves the source code, uses Apache Maven to build the application, creates the WAR artifact, builds a Docker image, pushes the image to Amazon Elastic Container Registry (Amazon ECR), and deploys the application to a Docker-enabled EC2 host.

The implementation uses separate responsibilities for the Jenkins CI/CD server and the Docker application host. Amazon ECR provides the private container image registry, while EC2 provides the compute environment where the application container runs. The supplied evidence shows the SampleApp application running with Java 17 and Tomcat 10 on port 8080.

2. Problem Statement

Manual Java application deployment requires repeated source retrieval, compilation, packaging, container creation, image delivery and runtime restart activities. These repeated operations can increase deployment effort and the possibility of configuration mistakes. This project addresses that workflow by automating the build and deployment process through Jenkins, Maven, Docker and Amazon ECR.

3. Objectives

- Automate Java application compilation, testing and packaging using Maven.
- Store application source and CI/CD configuration in GitHub.
- Trigger Jenkins through a GitHub webhook.
- Generate a deployable WAR file from the Java project.
- Build a reproducible Docker image containing the Java/Tomcat application.
- Store the image in a private Amazon ECR repository.
- Deploy and run the image on an AWS EC2 Docker host.
- Serve the Java web application through Tomcat on port 8080.
- Provide visible Jenkins stages and console logs for pipeline verification.
- Document implementation evidence, troubleshooting and final validation.

4. Technologies and Tools

Technology / Service	Purpose	Role in Project
GitHub	Source control	Stores Java application, pom.xml, Dockerfile and Jenkinsfile
Jenkins	CI/CD automation	Orchestrates checkout, build, image creation, registry push and deployment
Apache Maven	Java build automation	Compiles/tests/packages the application and generates the WAR
Java 17	Java build/runtime	Build/runtime environment shown in the implementation
Docker	Containerization	Builds and runs the application container
Amazon ECR	Container registry	Stores the java-webapp Docker image
Amazon EC2	Compute	Hosts Jenkins and the Docker application environment
Apache Tomcat 10	Java web server	Runs the packaged Java web application
AWS CLI	AWS automation	Used for AWS/ECR operations from the terminal
GitHub Webhook	Event trigger	Connects GitHub push events to Jenkins

5. Prerequisites

- AWS account with permissions required for EC2, ECR and related configuration.
- Jenkins EC2 instance running Amazon Linux 2023.
- Docker-enabled EC2 host for application deployment.
- Java and Maven available for the Java build.
- Git and a GitHub repository containing the Java application.
- Valid pom.xml for Maven packaging.
- Dockerfile capable of packaging the WAR into a Tomcat image.
- Jenkinsfile defining the CI/CD pipeline stages.
- Security-group rules allowing the required Jenkins and application ports.

6. System Architecture

The implementation follows a source-to-runtime CI/CD path. A developer pushes code to GitHub. GitHub sends a webhook event to Jenkins. Jenkins checks out the project, uses Maven to create the WAR artifact, builds the Docker image and pushes it to Amazon ECR. The application EC2 host obtains the image, runs the container and exposes Tomcat on port 8080.

End-to-End AWS CI/CD Pipeline: Java-Tomcat Application

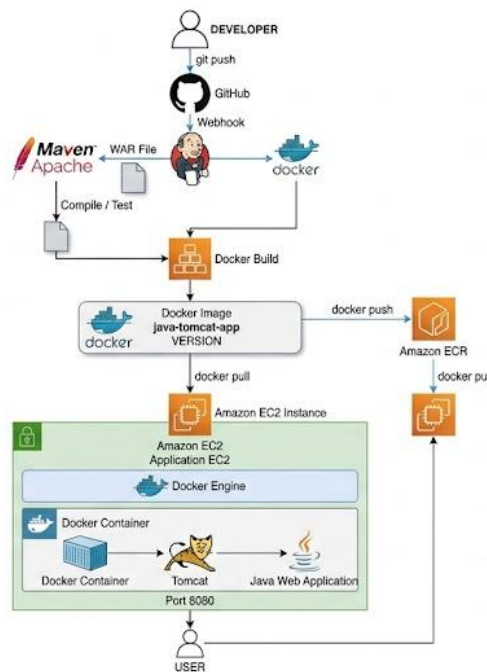


Figure 1. Complete Level 5 system architecture supplied for the project.

6.1 Architecture Flow

Step	Flow	Purpose
1	Developer → GitHub	Application changes are committed and pushed.
2	GitHub → Jenkins	Webhook notifies Jenkins about the repository event.
3	Jenkins → Checkout	Jenkins retrieves the application source.
4	Jenkins → Maven	Maven compiles/tests/packages the Java application into a WAR.
5	Jenkins → Docker	The WAR/application is packaged into a Docker image.
6	Jenkins → Amazon ECR	The Docker image is tagged and pushed to the private repository.
7	Amazon ECR → Application EC2	The application host authenticates and retrieves the image.
8	EC2 → Docker → Tomcat	The container runs Tomcat 10 and the Java web application.
9	User → EC2 :8080	The deployed SampleApp is accessed through the configured application endpoint.

7. Complete CI/CD Workflow

- Developer commits and pushes application changes to GitHub.
- GitHub webhook sends the push event to Jenkins.
- Jenkins starts the java-webapp-cicd Pipeline job.
- Jenkins checks out the source code.
- Maven builds the Java application and generates the WAR artifact.
- Jenkins builds the Docker image using the Dockerfile.
- The image is tagged for the Amazon ECR java-webapp repository.
- Jenkins authenticates to ECR and pushes the image.
- The deployment stage connects to the Docker EC2 host and retrieves the image.
- The existing application container is replaced/restarted according to the deployment configuration.
- Tomcat starts inside the container and serves the Java web application.
- The user verifies the deployed SampleApp through the application endpoint on port 8080.

8. Project Components

Component	Description
GitHub Repository	Central source-control location for the Java application and CI/CD files.
pom.xml	Defines Maven project metadata, dependencies and build/package behavior.
Dockerfile	Defines how the runtime image is assembled and how the WAR is placed into Tomcat.
Jenkinsfile	Defines the automated pipeline stages for checkout, build, Docker image creation, ECR push and deployment.
Jenkins EC2	Runs Jenkins and executes the CI/CD pipeline.
Docker EC2	Provides Docker Engine and the runtime environment for the application container.
Amazon ECR	Private registry used to store the java-webapp container image.
Tomcat Container	Runs the packaged Java web application and serves it on port 8080.

9. Implementation Procedure

The following 25 steps are presented in the chronological order. Each instruction/action is placed before its corresponding image.

Step 1 – Verify the Java web application source repository

Instruction / Action: The GitHub repository for the SampleApp Java web application is opened and verified. The repository is the source-control location for the application and CI/CD configuration files.

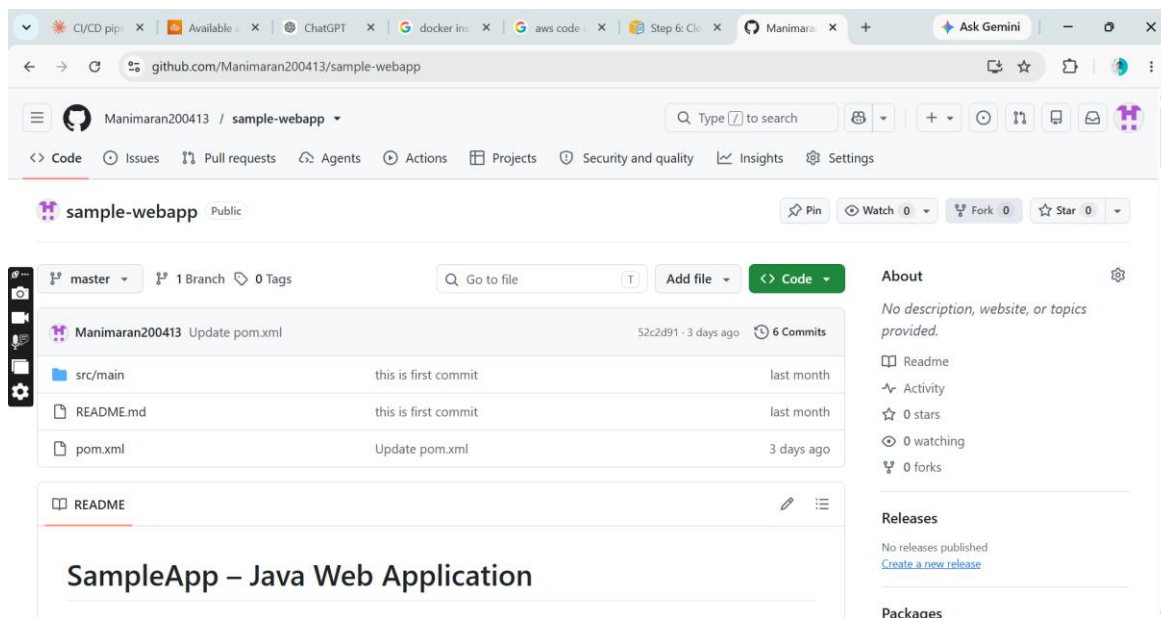


Figure 2. Verify the Java web application source repository.

Step 2 – Create/Verify the Jenkins EC2 instance

Instruction / Action: The AWS EC2 console shows the running Jenkins server in the Mumbai (ap-south-1) region. This instance is used as the CI/CD automation server.

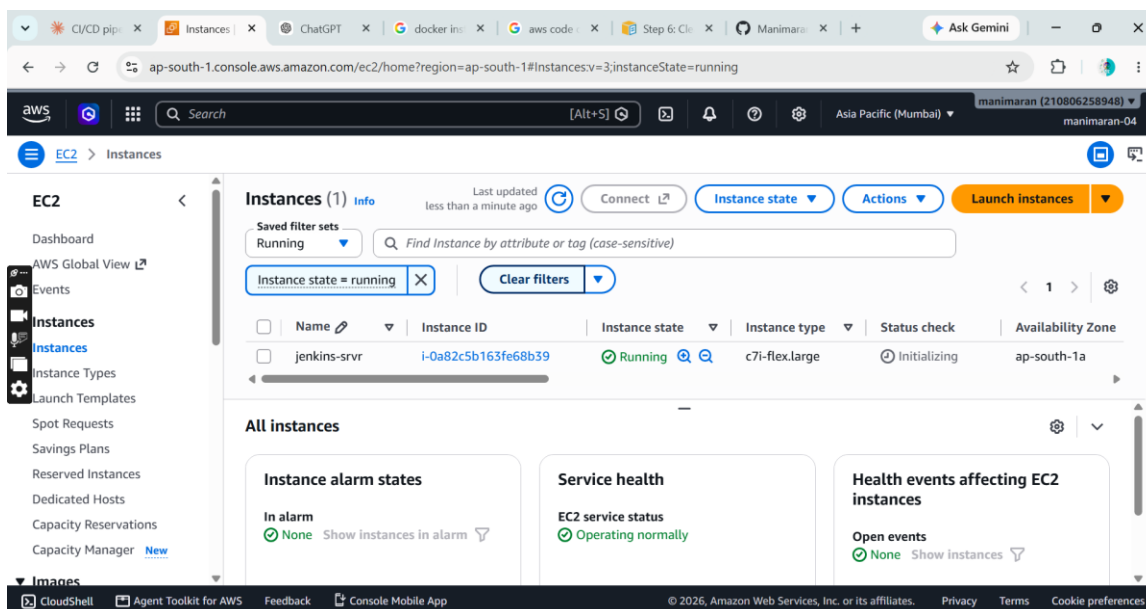
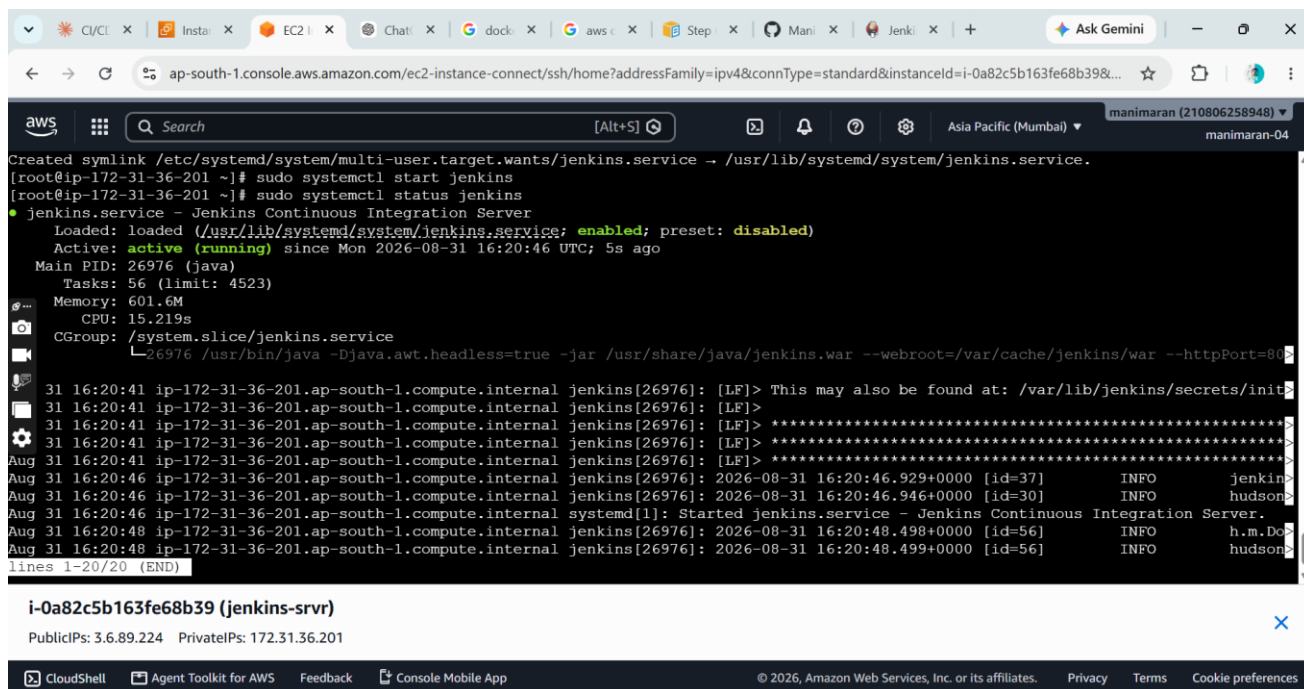


Figure 3. Create/Verify the Jenkins EC2 instance.

Step 3 – Enable and verify the Jenkins service

Instruction / Action: The EC2 terminal shows Jenkins being enabled through systemd. The service is configured to start automatically and is prepared for pipeline execution.



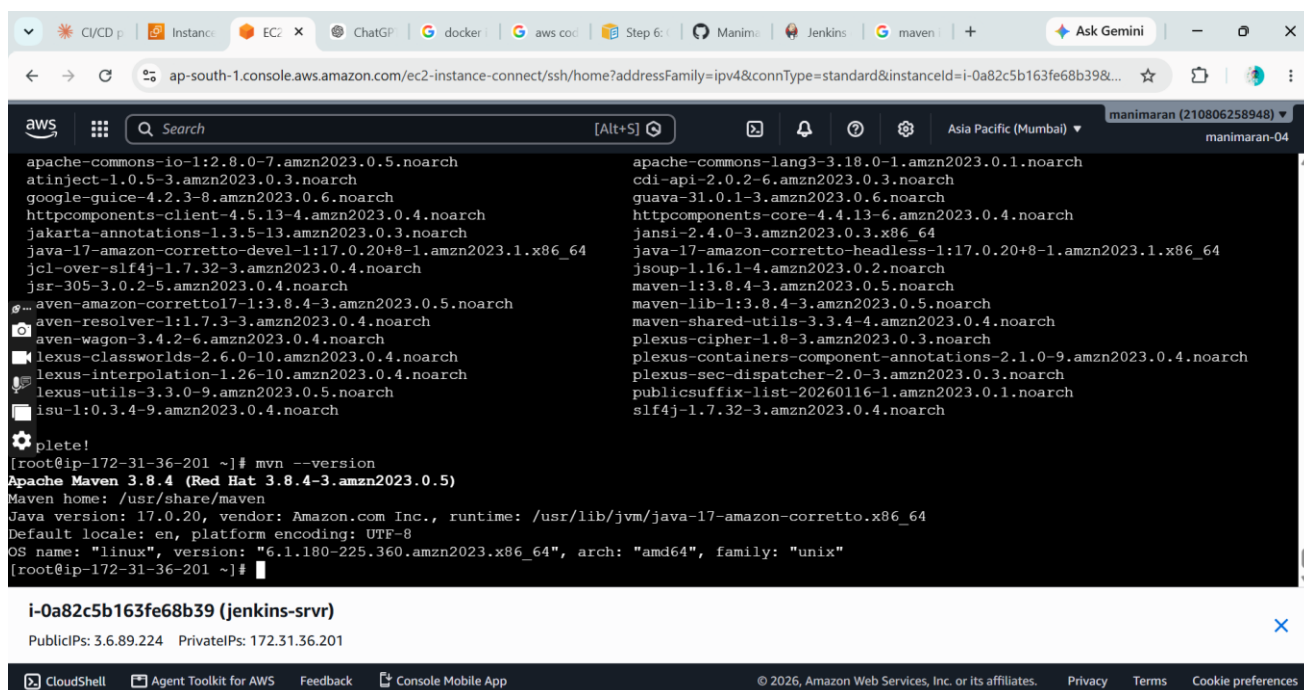
```
Created symlink /etc/systemd/system/multi-user.target.wants/jenkins.service -> /usr/lib/systemd/system/jenkins.service.
[root@ip-172-31-36-201 ~]# sudo systemctl start jenkins
[root@ip-172-31-36-201 ~]# sudo systemctl status jenkins
jenkins.service - Jenkins Continuous Integration Server
Loaded: loaded (/usr/lib/systemd/system/jenkins.service; enabled; preset: disabled)
Active: active (running) since Mon 2026-08-31 16:20:46 UTC; 5s ago
Main PID: 26976 (java)
Tasks: 56 (limit: 4523)
Memory: 601.6M
CPU: 15.219s
CGroup: /system.slice/jenkins.service
└─26976 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/java/jenkins.war --webroot=/var/cache/jenkins/war --httpPort=8080

31 16:20:41 ip-172-31-36-201.ap-south-1.compute.internal jenkins[26976]: [LF]> This may also be found at: /var/lib/jenkins/secrets/init
31 16:20:41 ip-172-31-36-201.ap-south-1.compute.internal jenkins[26976]: [LF]>
31 16:20:41 ip-172-31-36-201.ap-south-1.compute.internal jenkins[26976]: [LF]> *****
31 16:20:41 ip-172-31-36-201.ap-south-1.compute.internal jenkins[26976]: [LF]> *****
Aug 31 16:20:46 ip-172-31-36-201.ap-south-1.compute.internal jenkins[26976]: 2026-08-31 16:20:46.929+0000 [id=37] INFO jenkins
Aug 31 16:20:46 ip-172-31-36-201.ap-south-1.compute.internal jenkins[26976]: 2026-08-31 16:20:46.946+0000 [id=30] INFO hudson
Aug 31 16:20:46 ip-172-31-36-201.ap-south-1.compute.internal systemd[1]: Started jenkins.service - Jenkins Continuous Integration Server.
Aug 31 16:20:48 ip-172-31-36-201.ap-south-1.compute.internal jenkins[26976]: 2026-08-31 16:20:48.498+0000 [id=56] INFO h.m.Do
Aug 31 16:20:48 ip-172-31-36-201.ap-south-1.compute.internal jenkins[26976]: 2026-08-31 16:20:48.499+0000 [id=56] INFO hudson
lines 1-20/20 (END)
```

Figure 4. Enable and verify the Jenkins service.

Step 4 – Verify Java/Maven-related packages

Instruction / Action: The terminal shows the Amazon Linux environment and installed Java/Maven-related packages required for building the Java web application.



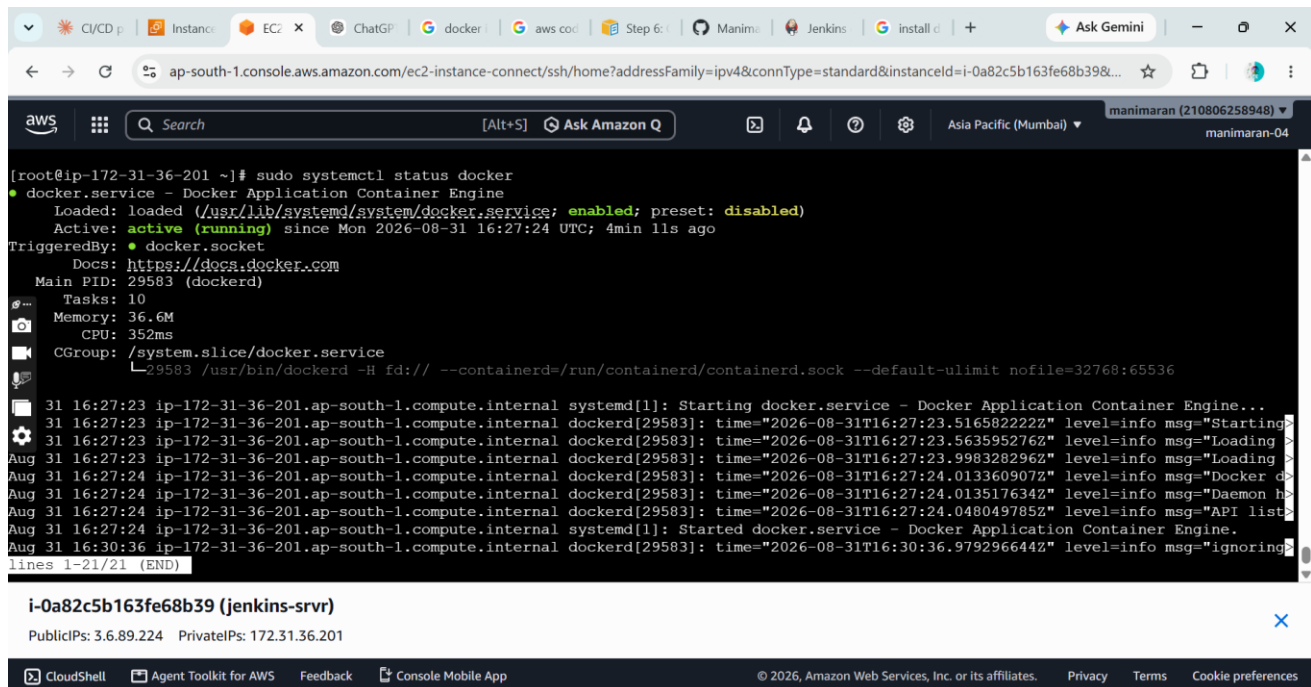
```
apache-commons-io-1:2.8.0-7.amzn2023.0.5.noarch
atinject-1.0.5-3.amzn2023.0.3.noarch
google-guice-4.2.3-8.amzn2023.0.6.noarch
httpcomponents-client-4.5.13-4.amzn2023.0.4.noarch
jakarta-annotations-1.3.5-13.amzn2023.0.3.noarch
java-17-amazon-corretto-devel-1:17.0.20+8-1.amzn2023.1.x86_64
jcl-over-slf4j-1.7.32-3.amzn2023.0.4.noarch
jsr-305-3.0.2-5.amzn2023.0.4.noarch
maven-3.8.4-3.amzn2023.0.5.noarch
maven-lib-1:3.8.4-3.amzn2023.0.5.noarch
maven-shared-utils-3.3.4-4.amzn2023.0.4.noarch
plexus-cipher-1.8-3.amzn2023.0.3.noarch
plexus-containers-component-annotations-2.1.0-9.amzn2023.0.4.noarch
plexus-sec-dispatcher-2.0-3.amzn2023.0.3.noarch
publicsuffix-list-20260116-1.amzn2023.0.1.noarch
slf4j-1.7.32-3.amzn2023.0.4.noarch

[root@ip-172-31-36-201 ~]# mvn --version
Apache Maven 3.8.4 (Red Hat 3.8.4-3.amzn2023.0.5)
Maven home: /usr/share/maven
Java version: 17.0.20, vendor: Amazon.com Inc., runtime: /usr/lib/jvm/java-17-amazon-corretto.x86_64
Default locale: en, platform encoding: UTF-8
OS name: "linux", version: "6.1.180-225.360.amzn2023.x86_64", arch: "amd64", family: "unix"
[root@ip-172-31-36-201 ~]#
```

Figure 5. Verify Java/Maven-related packages.

Step 5 – Verify Docker Engine

Instruction / Action: The Docker host terminal confirms that the Docker service is present and active. Docker is required to build and run the containerized Java/Tomcat application.



The screenshot shows the AWS CloudShell interface with a terminal window. The terminal output displays the status of the Docker service using the command `sudo systemctl status docker`. The output indicates that the Docker service is loaded and active (running) since Mon 2026-08-31 16:27:24 UTC. It also shows the Docker socket path, documentation link, and main PID. Below this, the terminal shows the Docker daemon logs, including the start of the Docker service, the loading of the daemon, and the start of the Docker daemon.

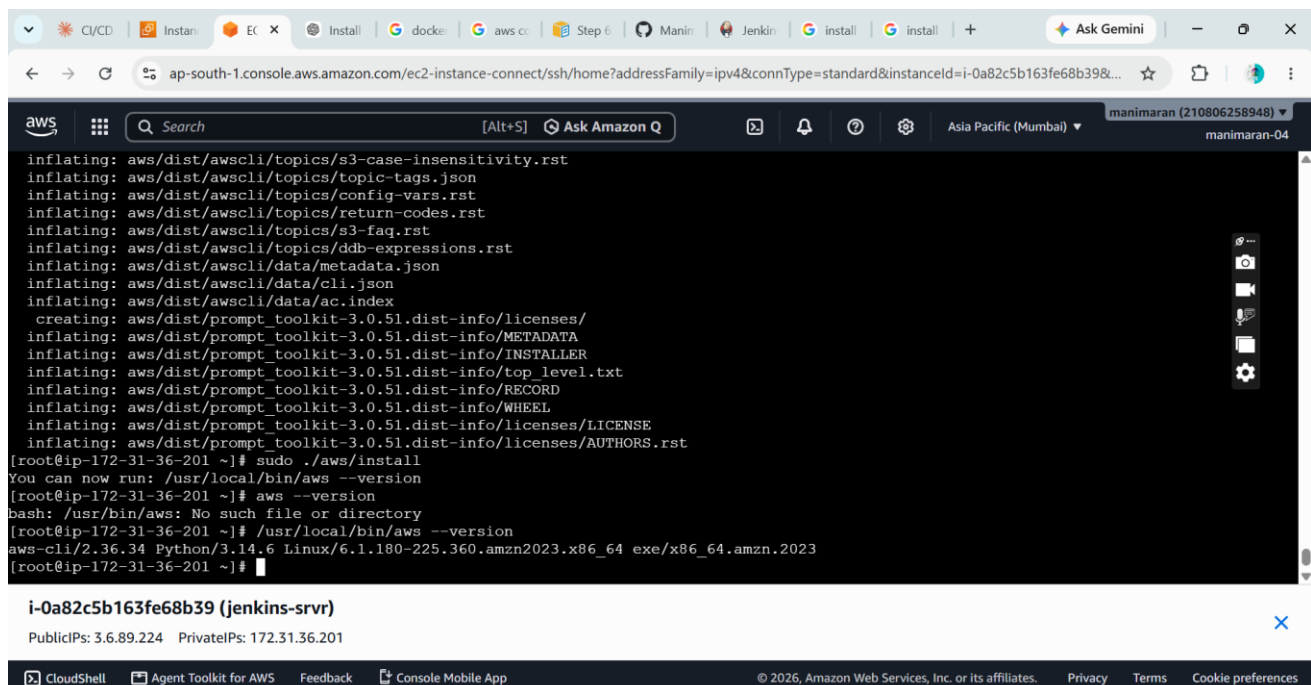
```
[root@ip-172-31-36-201 ~]# sudo systemctl status docker
● docker.service - Docker Application Container Engine
   Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: disabled)
   Active: active (running) since Mon 2026-08-31 16:27:24 UTC; 4min 11s ago
 TriggeredBy: ● docker.socket
   Docs: https://docs.docker.com
   Main PID: 29583 (dockerd)
      Tasks: 10
     Memory: 36.6M
        CPU: 352ms
   CGroup: /system.slice/docker.service
           └─29583 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock --default-ulimit nfile=32768:65536

Aug 31 16:27:23 ip-172-31-36-201.ap-south-1.compute.internal systemd[1]: Starting docker.service - Docker Application Container Engine...
Aug 31 16:27:23 ip-172-31-36-201.ap-south-1.compute.internal dockerd[29583]: time="2026-08-31T16:27:23.516582222Z" level=info msg="Starting
Aug 31 16:27:23 ip-172-31-36-201.ap-south-1.compute.internal dockerd[29583]: time="2026-08-31T16:27:23.563595276Z" level=info msg="Loading
Aug 31 16:27:23 ip-172-31-36-201.ap-south-1.compute.internal dockerd[29583]: time="2026-08-31T16:27:23.998328296Z" level=info msg="Loading
Aug 31 16:27:24 ip-172-31-36-201.ap-south-1.compute.internal dockerd[29583]: time="2026-08-31T16:27:24.013360907Z" level=info msg="Docker d
Aug 31 16:27:24 ip-172-31-36-201.ap-south-1.compute.internal dockerd[29583]: time="2026-08-31T16:27:24.013517634Z" level=info msg="Daemon h
Aug 31 16:27:24 ip-172-31-36-201.ap-south-1.compute.internal dockerd[29583]: time="2026-08-31T16:27:24.048049785Z" level=info msg="API list
Aug 31 16:27:24 ip-172-31-36-201.ap-south-1.compute.internal systemd[1]: Started docker.service - Docker Application Container Engine.
Aug 31 16:30:36 ip-172-31-36-201.ap-south-1.compute.internal dockerd[29583]: time="2026-08-31T16:30:36.979296644Z" level=info msg="ignoring
lines 1-21/21 (END)
```

Figure 6. Verify Docker Engine.

Step 6 – Install/verify AWS CLI

Instruction / Action: The terminal shows AWS CLI installation activity and extracted AWS CLI files. AWS CLI is used for AWS service operations, including Amazon ECR.



The screenshot shows the AWS CloudShell interface with a terminal window. The terminal output displays the installation of the AWS CLI. It shows the inflating of various files and the creation of the prompt toolkit. The installation is completed, and the user is prompted to run the command `aws --version` to verify the installation. The output shows the version of the AWS CLI installed.

```
inflating: aws/dist/awscli/topics/s3-case-insensitivity.rst
inflating: aws/dist/awscli/topics/topic-tags.json
inflating: aws/dist/awscli/topics/config-vars.rst
inflating: aws/dist/awscli/topics/return-codes.rst
inflating: aws/dist/awscli/topics/s3-faq.rst
inflating: aws/dist/awscli/topics/db-expressions.rst
inflating: aws/dist/awscli/data/metadata.json
inflating: aws/dist/awscli/data/cli.json
inflating: aws/dist/awscli/data/ac.index
creating: aws/dist/prompt_toolkit-3.0.51.dist-info/licenses/
inflating: aws/dist/prompt_toolkit-3.0.51.dist-info/METADATA
inflating: aws/dist/prompt_toolkit-3.0.51.dist-info/INSTALLER
inflating: aws/dist/prompt_toolkit-3.0.51.dist-info/top_level.txt
inflating: aws/dist/prompt_toolkit-3.0.51.dist-info/RECORD
inflating: aws/dist/prompt_toolkit-3.0.51.dist-info/WHEEL
inflating: aws/dist/prompt_toolkit-3.0.51.dist-info/licenses/LICENSE
inflating: aws/dist/prompt_toolkit-3.0.51.dist-info/licenses/AUTHORS.rst
[root@ip-172-31-36-201 ~]# sudo ./aws/install
You can now run: /usr/local/bin/aws --version
[root@ip-172-31-36-201 ~]# aws --version
bash: /usr/bin/aws: No such file or directory
[root@ip-172-31-36-201 ~]# /usr/local/bin/aws --version
aws-cli/2.36.34 Python/3.14.6 Linux/6.1.180-225.360.amzn2023.x86_64 exe/x86_64.amzn.2023
[root@ip-172-31-36-201 ~]#
```

Figure 7. Install/verify AWS CLI.

Step 7 – Verify/Unlock the Jenkins Page

Instruction / Action: Copy and paste the public IP address of Jenkins server on internet. It shows, The Jenkins initial setup page is displayed. Jenkins requests the initial administrator password stored on the server before completing the first-time setup.

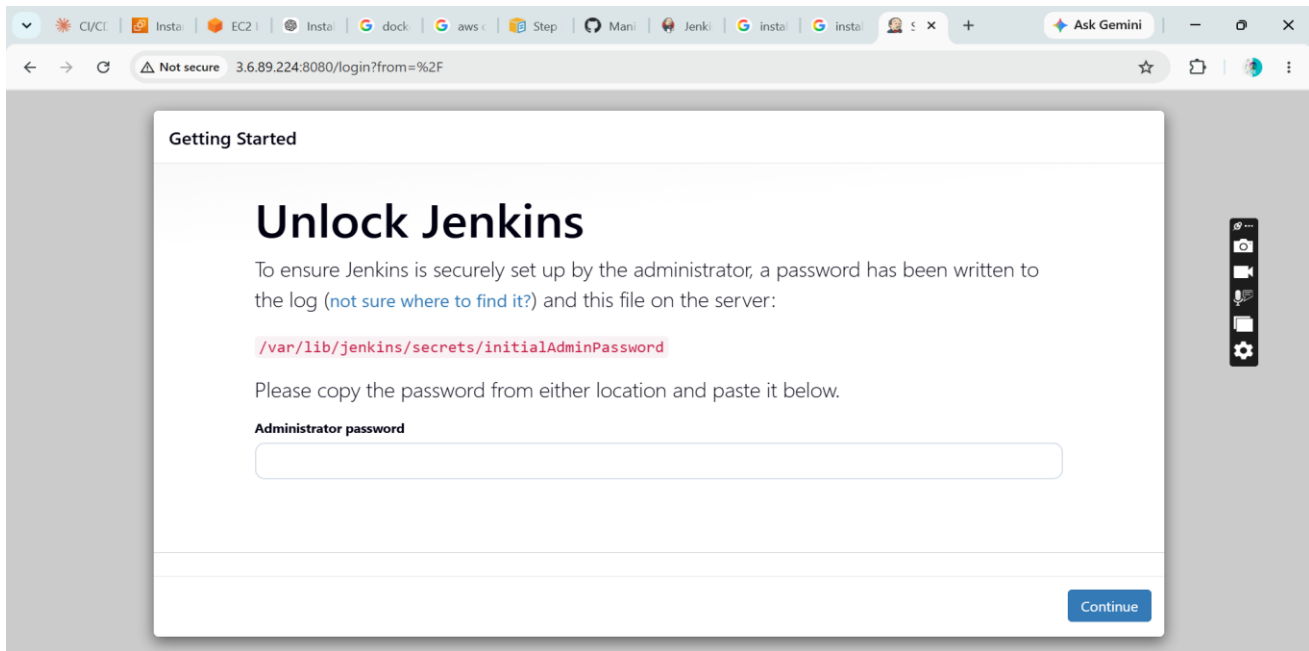


Figure 8. Verify/Unlock Jenkins.

Step 8 – Create/verify the Amazon ECR repository

Instruction / Action: The AWS CLI is used to create the private java-webapp repository. The implementation evidence also shows an initial endpoint/region input issue, which was corrected before the repository was successfully created.

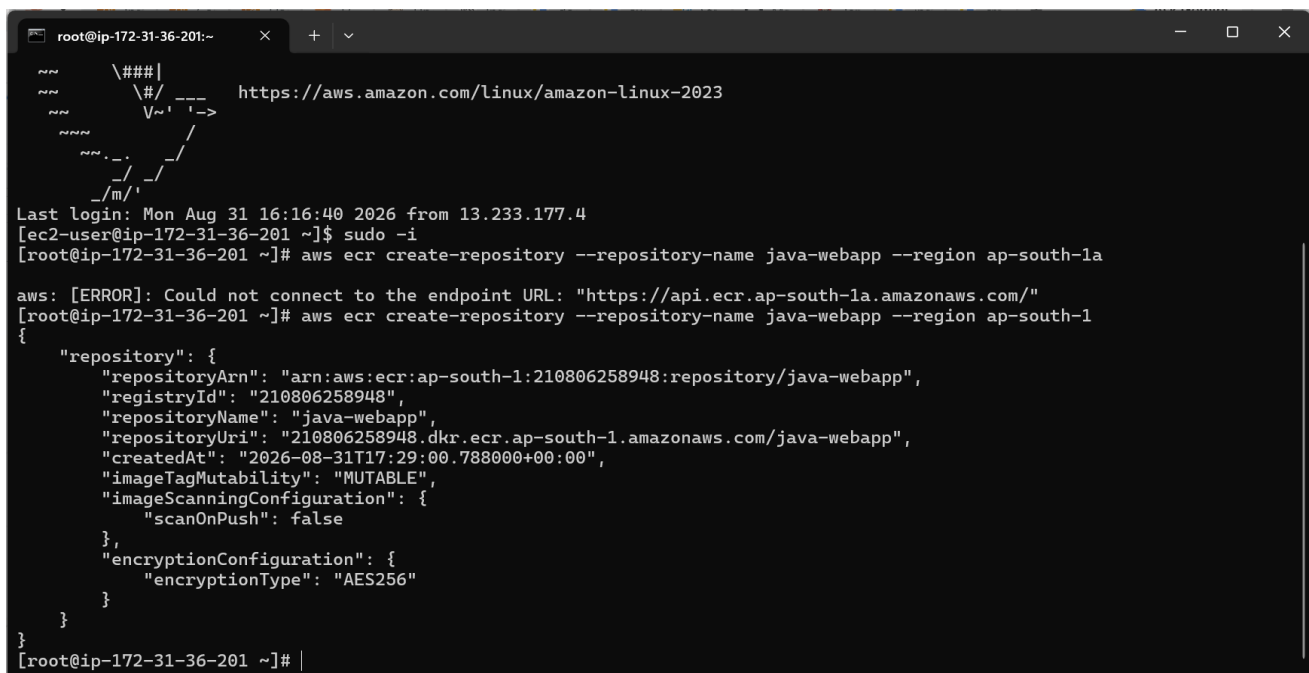


Figure 9. Step 8 – Create/verify the Amazon ECR repository.

Step 9 – Verify Amazon ECR repository

Instruction / Action: The Amazon ECR console shows the private java-webapp repository in ap-south-1, confirming that the container registry is available for the project.

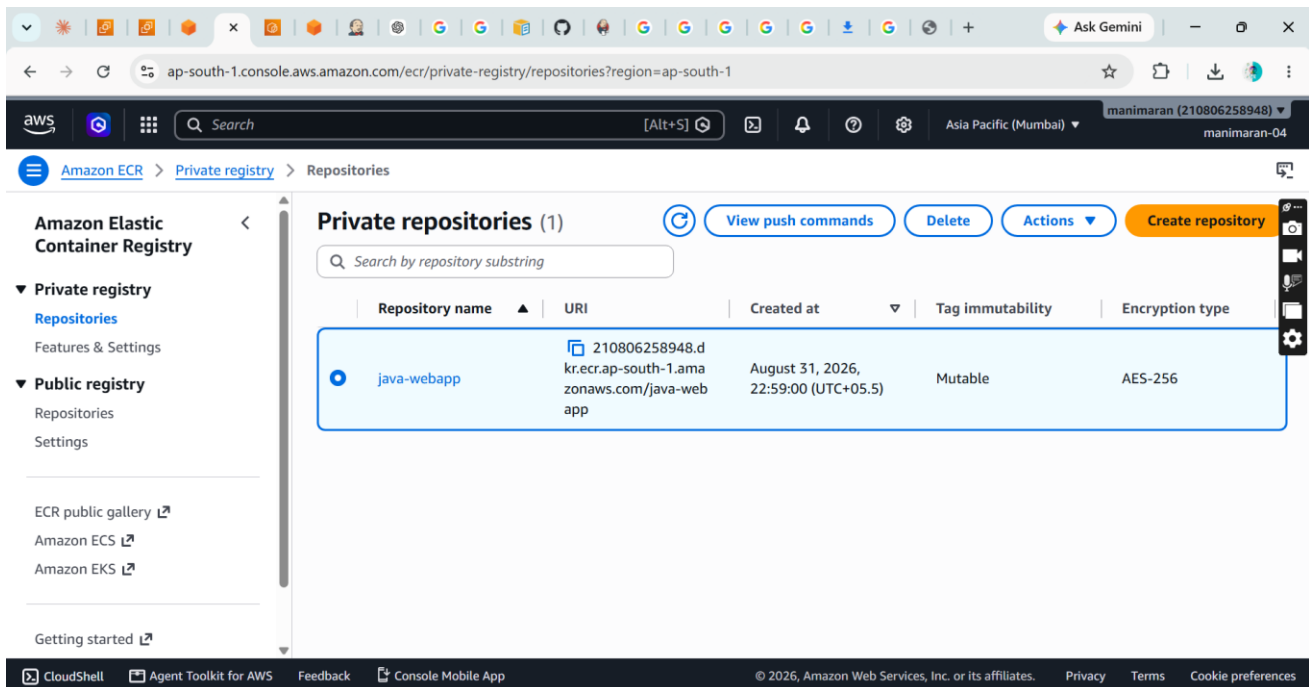


Figure 10. Verify Amazon ECR repository.

Step 10 – Synchronize the Git repository

Instruction / Action: The Git terminal shows merge/synchronization activity with the remote repository before the required CI/CD files are committed and pushed.

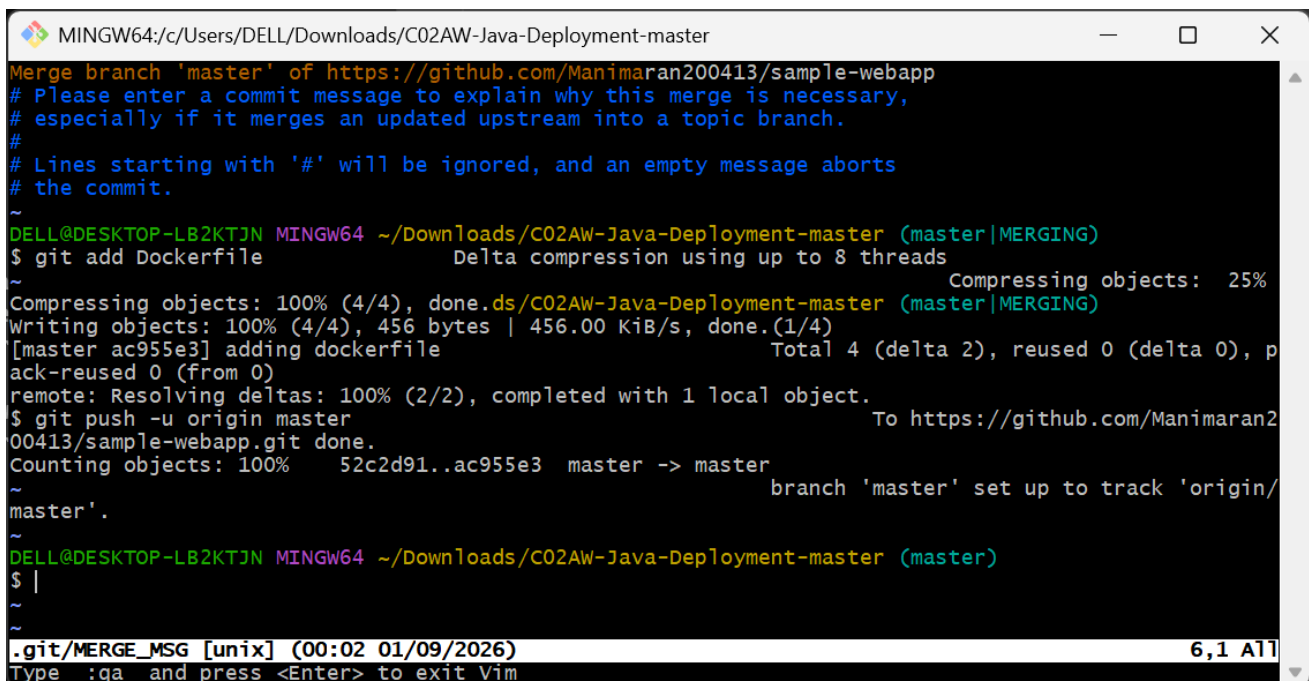


Figure 11. Synchronize the Git repository.

Step 11 – Verify the GitHub repository after changes

Instruction / Action: The GitHub repository is refreshed and the project contents are visible, confirming that the application source and supporting files are available in the remote repository.

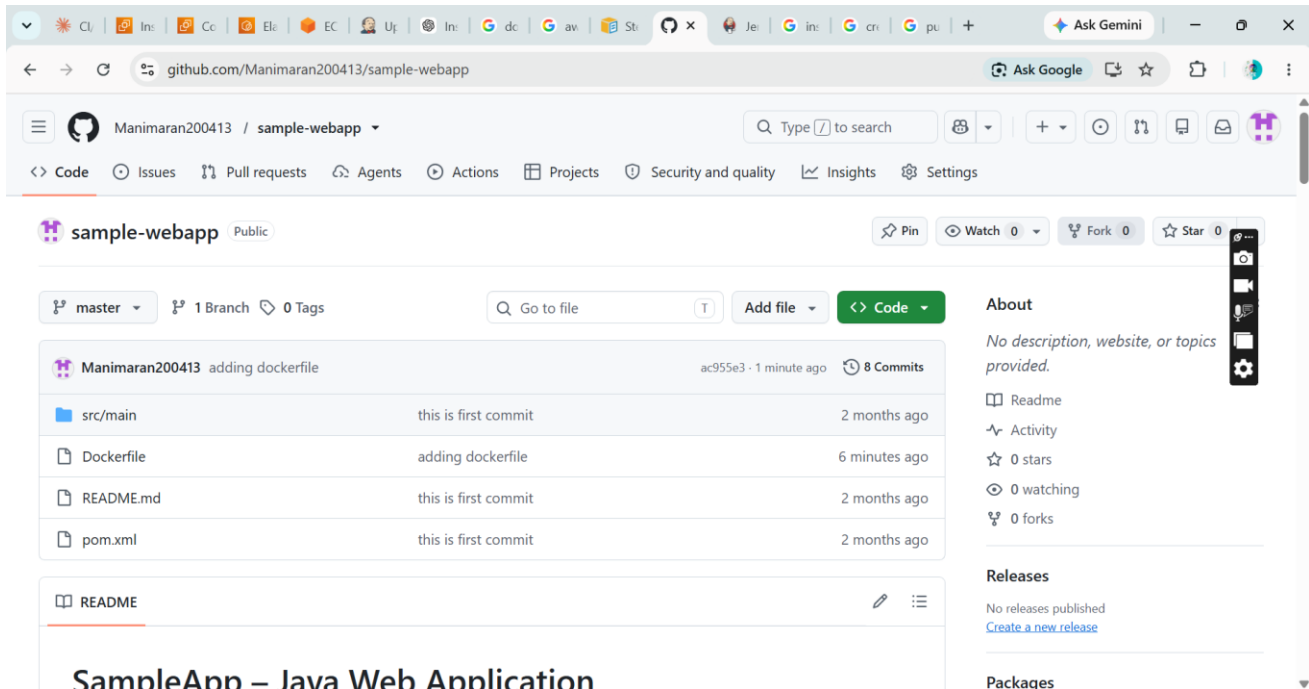


Figure 12. Verify the GitHub repository after changes.

Step 12 – Commit the Jenkinsfile

Instruction / Action: The local Git terminal records a commit adding the Jenkinsfile. The Jenkinsfile defines the automated CI/CD stages.

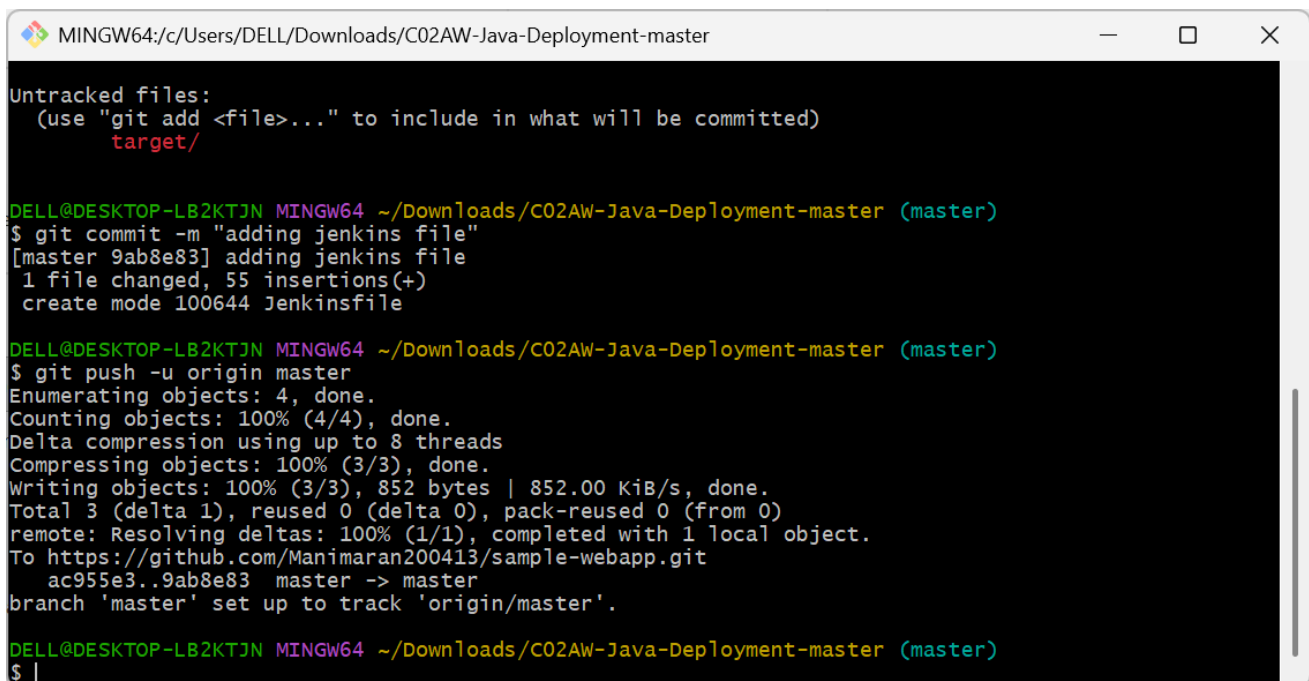


Figure 13. Commit the Jenkinsfile.

Step 13 – Verify project files in GitHub

Instruction / Action: The GitHub repository contents are checked again after the Jenkinsfile change, confirming the project files are version controlled.

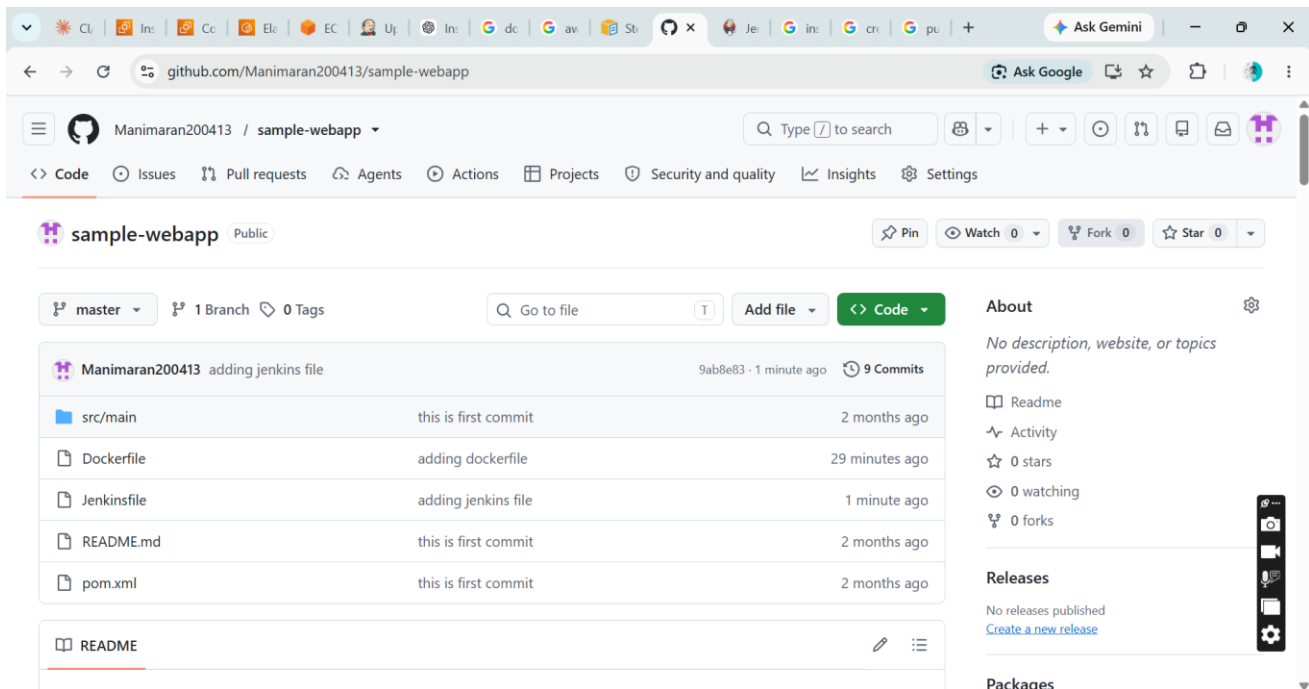


Figure 14. Verify project files in GitHub.

Step 14 – Create & Verify Docker host service status

Instruction / Action: Create the docker host server and The application/Docker EC2 terminal confirms Docker Engine is active and available for container operations.

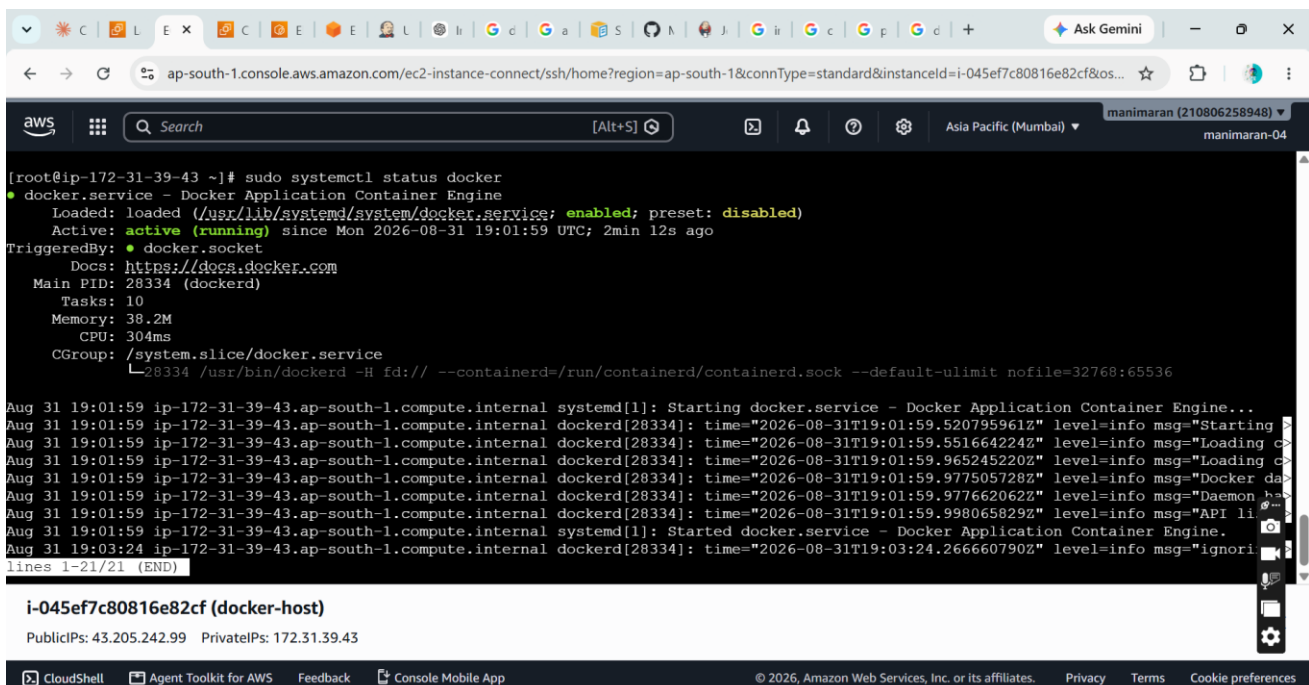
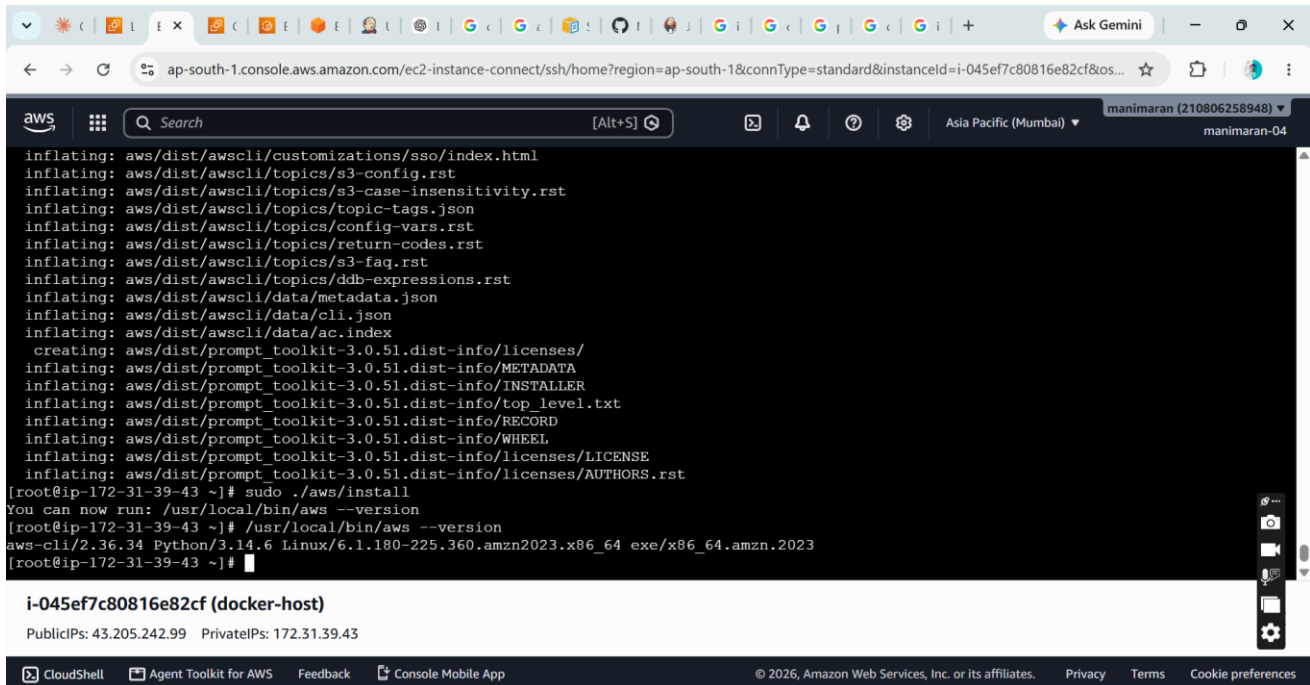


Figure 15. Create & Verify Docker host service status.

Step 15 – Verify AWS CLI/runtime setup on the application host

Instruction / Action: The terminal shows AWS CLI setup activity on the application EC2 host, supporting ECR authentication and image retrieval.



```
inflatng: aws/dist/awscli/customizations/sso/index.html
inflatng: aws/dist/awscli/topics/s3-config.rst
inflatng: aws/dist/awscli/topics/s3-case-insensitivity.rst
inflatng: aws/dist/awscli/topics/topic-tags.json
inflatng: aws/dist/awscli/topics/config-vars.rst
inflatng: aws/dist/awscli/topics/return-codes.rst
inflatng: aws/dist/awscli/topics/s3-faq.rst
inflatng: aws/dist/awscli/topics/db-expressions.rst
inflatng: aws/dist/awscli/data/metadata.json
inflatng: aws/dist/awscli/data/cli.json
inflatng: aws/dist/awscli/data/ac.index
creating: aws/dist/prompt_toolkit-3.0.51.dist-info/licenses/
inflatng: aws/dist/prompt_toolkit-3.0.51.dist-info/METADATA
inflatng: aws/dist/prompt_toolkit-3.0.51.dist-info/INSTALLER
inflatng: aws/dist/prompt_toolkit-3.0.51.dist-info/top_level.txt
inflatng: aws/dist/prompt_toolkit-3.0.51.dist-info/RECORD
inflatng: aws/dist/prompt_toolkit-3.0.51.dist-info/WHEEL
inflatng: aws/dist/prompt_toolkit-3.0.51.dist-info/licenses/LICENSE
inflatng: aws/dist/prompt_toolkit-3.0.51.dist-info/licenses/AUTHORS.rst
[root@ip-172-31-39-43 ~]# sudo ./aws/install
You can now run: /usr/local/bin/aws --version
[root@ip-172-31-39-43 ~]# /usr/local/bin/aws --version
aws-cli/2.36.34 Python/3.14.6 Linux/6.1.180-225.360.amzn2023.x86_64 exe/x86_64.amzn.2023
[root@ip-172-31-39-43 ~]#
```

i-045ef7c80816e82cf (docker-host)
PublicIPs: 43.205.242.99 PrivateIPs: 172.31.39.43

Figure 16. Verify AWS CLI/runtime setup on the application host.

Step 16 – Create the Jenkins Pipeline job

Instruction / Action: The Jenkins New Item page is used to create the java-webapp-cicd Pipeline job.

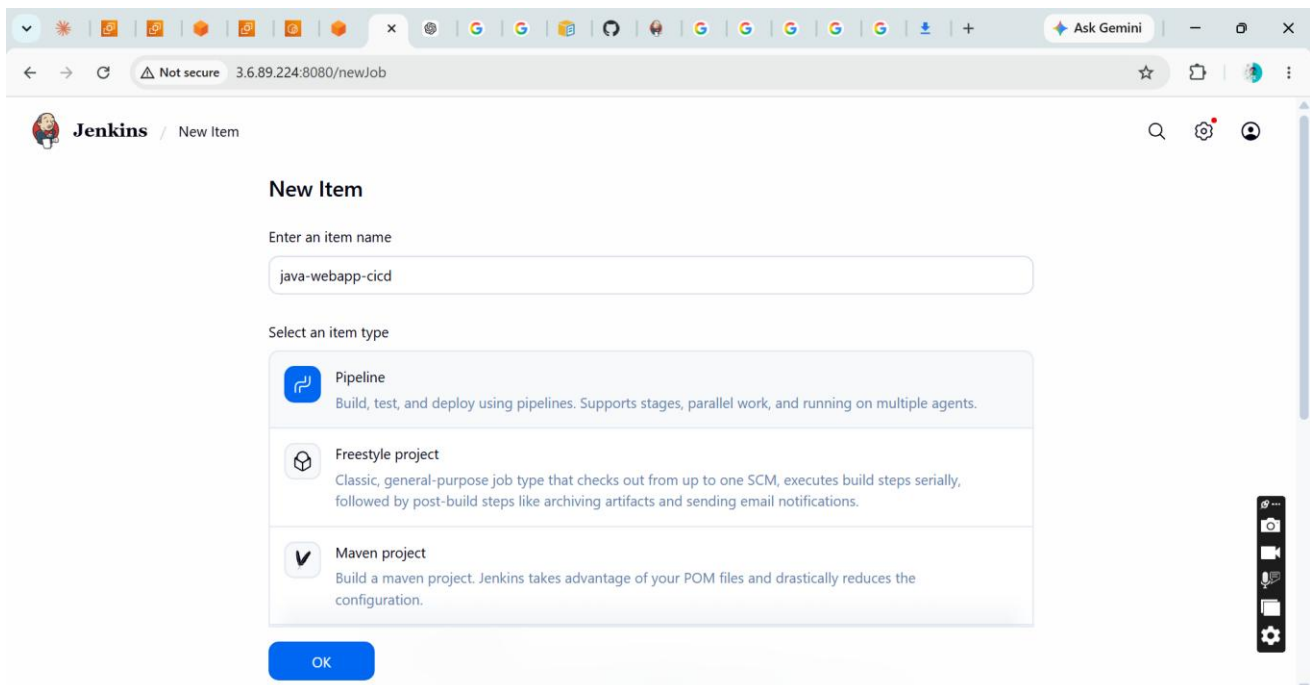


Figure 17. Create the Jenkins Pipeline job.

Step 17 – Configure the GitHub webhook

Instruction / Action: The GitHub Webhooks page confirms that a webhook was created for Jenkins and a test ping was sent. This connects repository push events with Jenkins.

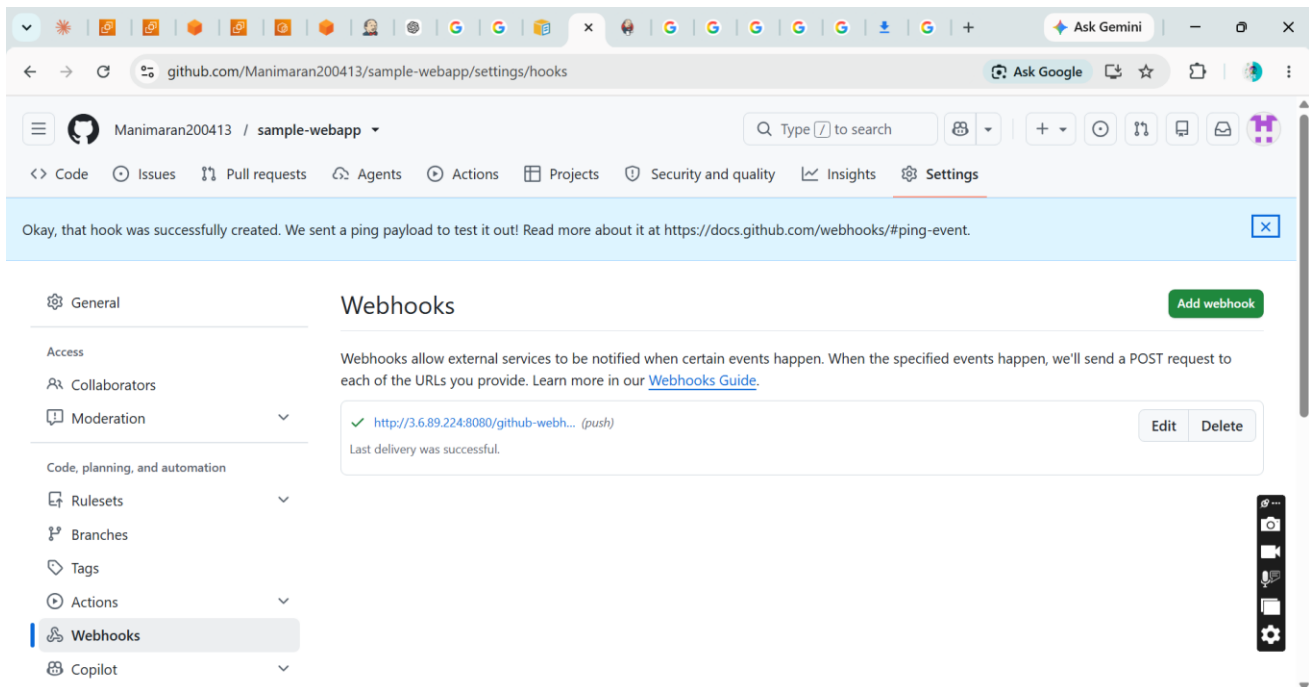


Figure 18. Configure the GitHub webhook.

Step 18 – Verify build environment packages

Instruction / Action: The Jenkins EC2 terminal shows installation/verification output for packages needed by the build environment.

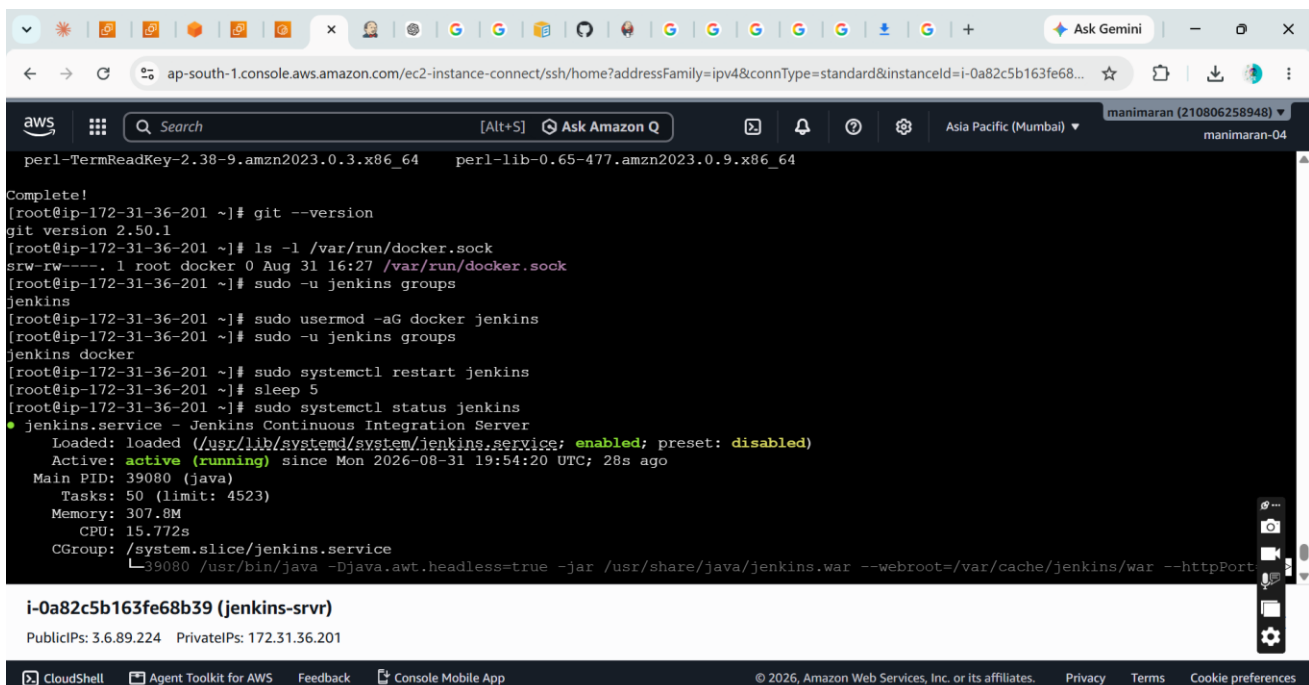


Figure 19. Verify build environment packages.

Step 19 – Review the Error pipeline execution

Instruction / Action: The Jenkins Stage View shows an early pipeline run where a stage did not complete successfully. Because WAR file name mismatch in the Dockerfile.so Maven couldn't able to build the java app.

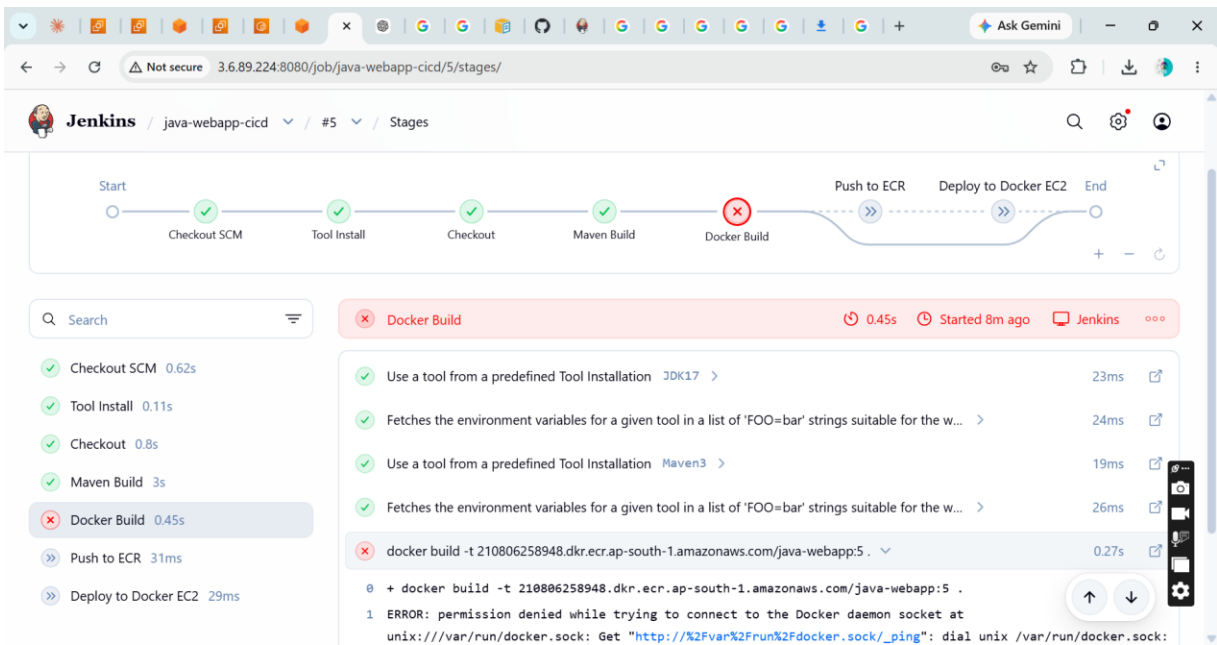


Figure 20. Review the first pipeline execution.

Step 20 – Verify the successful pipeline stages

Instruction / Action: After correcting the mistake in Dockerfile.The Jenkins Stage View shows the java-webapp-cicd pipeline with the major stages progressing through checkout, tool setup, Maven build, Docker build, push to ECR and deployment.

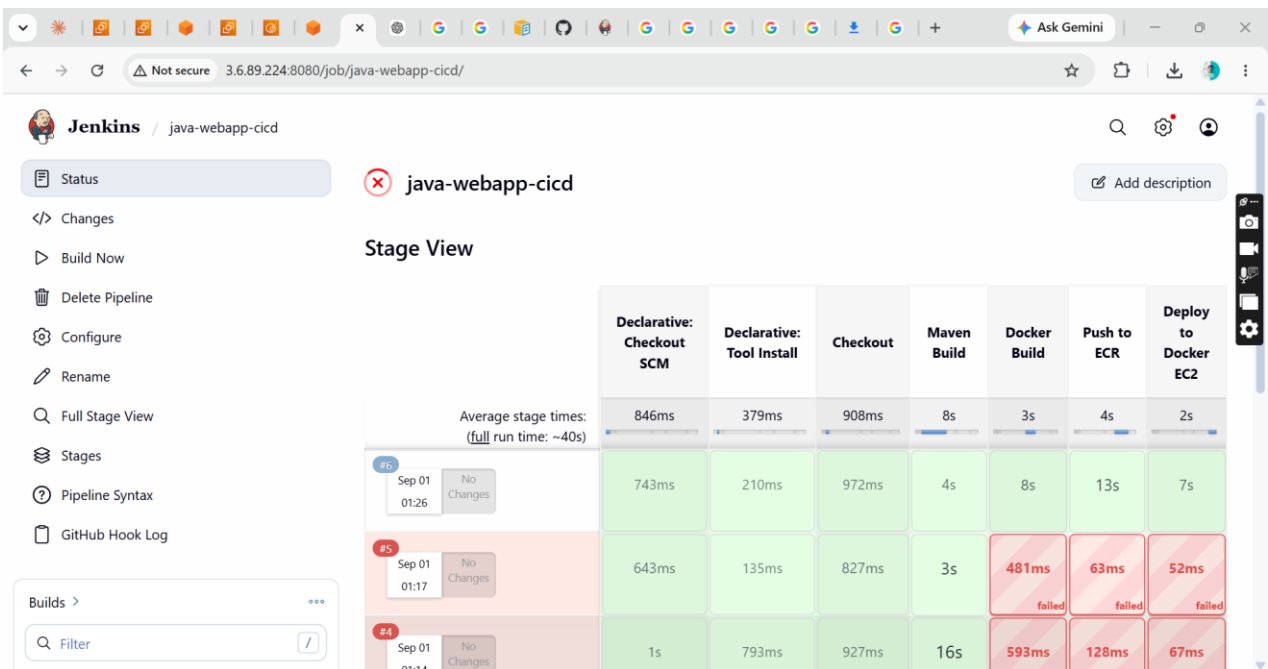
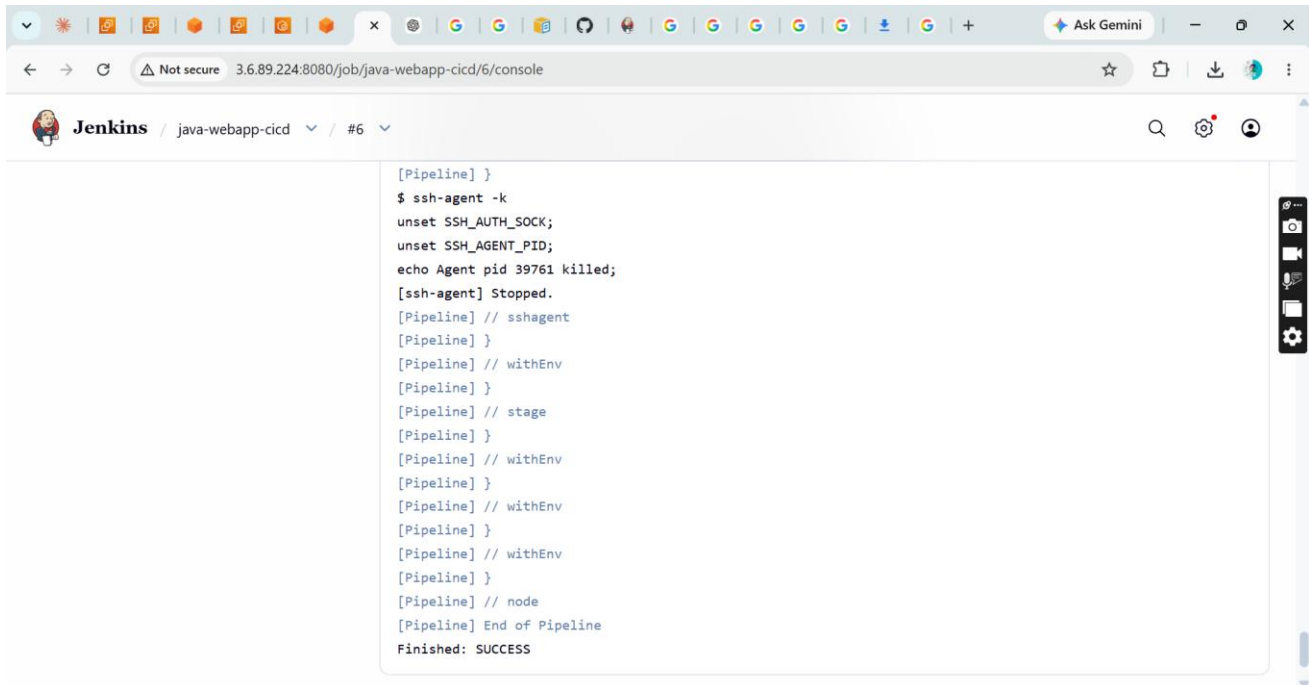


Figure 21. Verify the successful pipeline stages.

Step 21 – Review Jenkins console output

Instruction / Action: The Jenkins console output shows the end of a pipeline execution, including SSH-agent cleanup and completion of the pipeline stages.

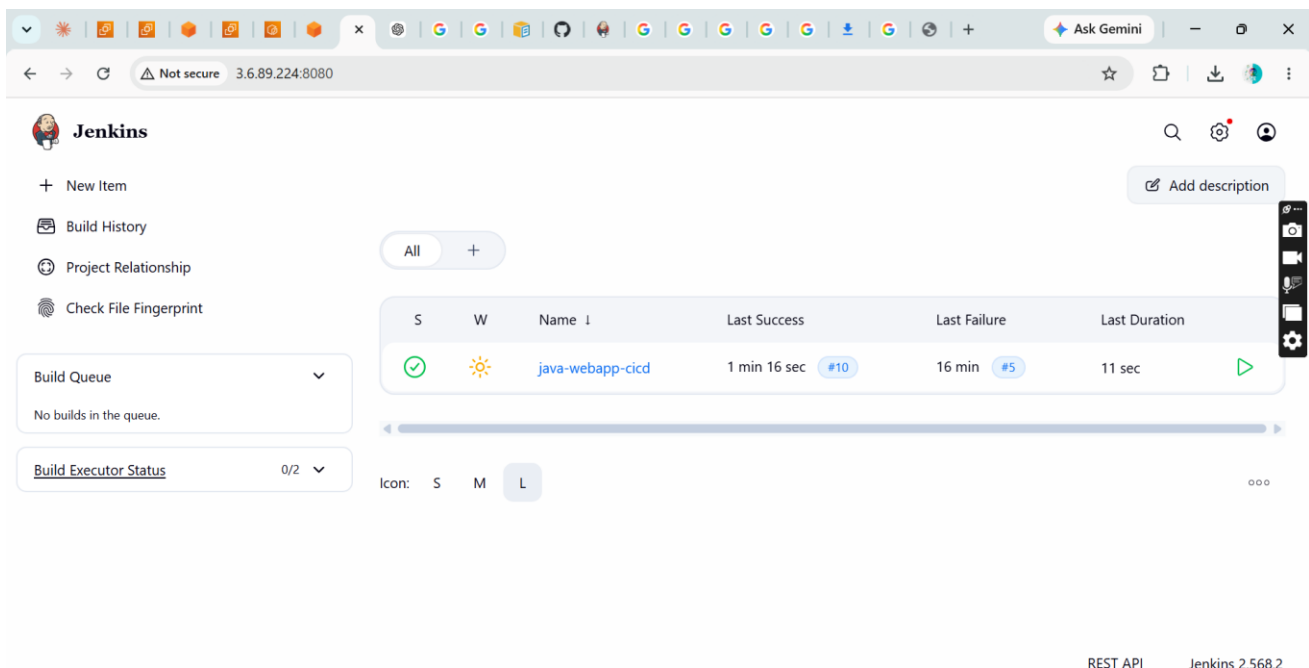


```
[Pipeline] }
$ ssh-agent -k
unset SSH_AUTH_SOCK;
unset SSH_AGENT_PID;
echo Agent pid 39761 killed;
[ssh-agent] Stopped.
[Pipeline] // sshagent
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Figure 21. Review Jenkins console output.

Step 22 – Verify Jenkins final build status

Instruction / Action: The Jenkins dashboard shows the java-webapp-cicd job with a recent successful build, confirming the pipeline is operational.



The Jenkins dashboard shows the 'java-webapp-cicd' job with a recent successful build. The build status is 'S' (Success) with a green checkmark icon. The build number is #10, and the last success time is 1 min 16 sec. The last failure time is 16 min #5, and the last duration is 11 sec. The build is currently in the queue.

S	W	Name	Last Success	Last Failure	Last Duration
✓	☀	java-webapp-cicd	1 min 16 sec #10	16 min #5	11 sec

Figure 23. Verify Jenkins final build status.

Step 23 – Attach the ECR IAM role to the Docker host

Instruction / Action: The EC2 console reports that the ec2-ecr role was successfully attached to the docker-host instance. This provides the instance identity used for AWS/ECR operations.

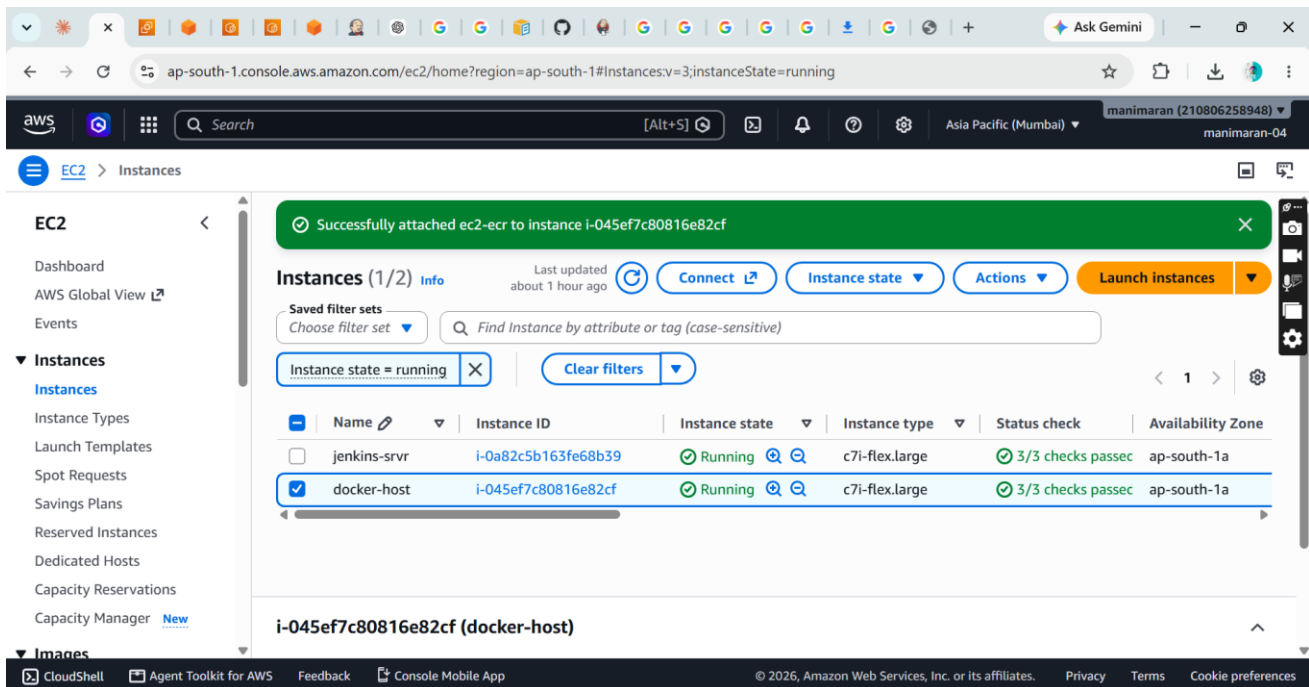


Figure 24. Attach the ECR IAM role to the Docker host.

Step 24 – Verify Docker/AWS CLI setup on the Docker host

Instruction / Action: The Docker host terminal shows AWS CLI installation/extraction output. This prepares the host to interact with Amazon ECR.

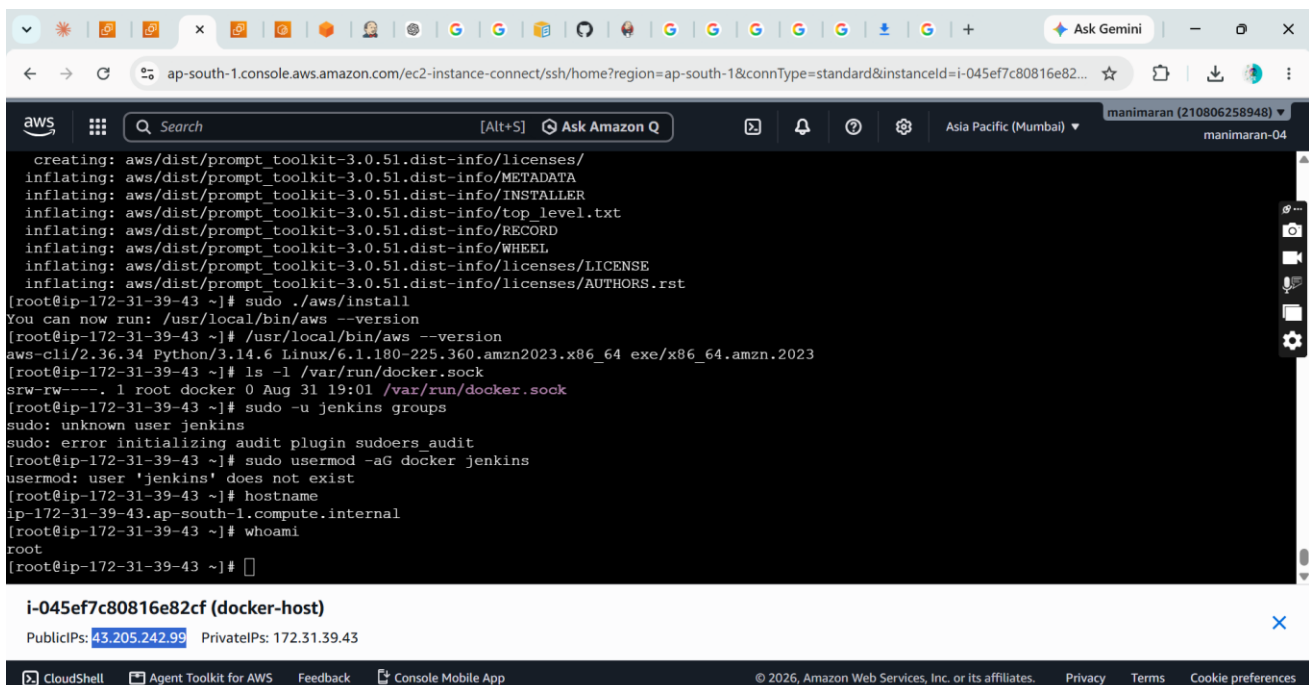


Figure 25. Verify Docker/AWS CLI setup on the Docker host.

Step 25 – Validate the deployed SampleApp

Instruction / Action: The browser displays the SampleApp home page through the EC2 public endpoint on port 8080. The page shows Java 17 and Tomcat 10 references.

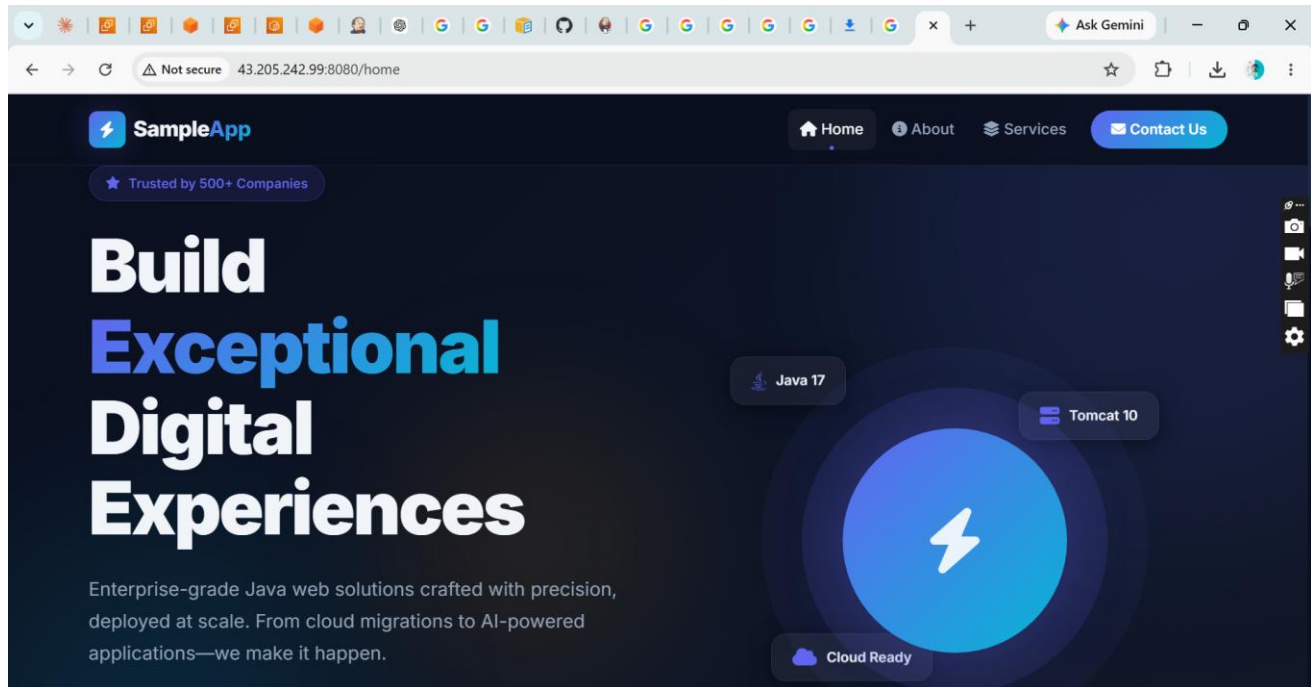


Figure 26. Validate the deployed SampleApp.

10. Error Handling and Resolution

How Errors are handled and resolved are described Below in following topics,

10.1 Amazon ECR region/endpoint input issue

During the initial ECR repository command, the terminal showed an endpoint connection error caused by an incorrect region value in the command. The repository operation was subsequently corrected and the java-webapp repository was created in ap-south-1.

10.2 Dockerfile WAR file mismatch issue

I found this error after run the Jenkins job build in cicd pipe line .It throwing the error like can't able to build the java app cause of Artifact mismatch .so after I found and correct the error eventually Jenkins job builds run successfully and the Maven build the java app WAR file after converting that into Docker image it was stored in AWS ECR repository.

10.3 Docker permission issue

The project implementation also encountered a Docker daemon permission problem in the Jenkins workflow. The issue was associated with Jenkins access to /var/run/docker.sock. The documented resolution was to ensure the Jenkins service account had Docker-group access and to restart Jenkins so the new group membership applied to the running service. The pipeline was then re-run and the Docker Build, ECR push and deployment stages completed.

10.4 General verification points

- Check Docker service status before running Docker commands.
- Check Docker socket permissions when Jenkins cannot connect to Docker.
- Verify the ECR repository name and AWS region before pushing images.
- Verify the EC2 instance has the required IAM/ECR permissions.
- Review Jenkins Stage View and console output to locate the failed stage.
- Re-run the pipeline after correcting the underlying configuration issue.

11. Testing and Validation

Test	Purpose	Observed Evidence	Status
GitHub repository	Source availability	Repository and project files visible	Verified
Jenkins service	CI availability	Jenkins service/UI available	Verified
Java/Maven environment	Build prerequisites	Java/Maven packages shown in terminal	Verified
Docker Engine	Container operations	Docker service shown active	Verified
AWS CLI	AWS/ECR operations	AWS CLI installation/version activity shown	Verified
ECR repository	Image registry	java-webapp private repository visible	Verified
GitHub webhook	Pipeline trigger	Webhook created and ping sent	Verified
Jenkins pipeline	CI/CD execution	Stage View and console evidence available	Verified
ECR IAM attachment	EC2-side AWS access	ec2-ecr successfully attached to docker-host	Verified
Application deployment	Runtime availability	SampleApp home page displayed on port 8080	Verified

12. Security Considerations

- Do not commit AWS access keys, secret keys, passwords, tokens or private-key material to GitHub.
- Treat the Jenkins initial administrator password as sensitive information.
- Prefer IAM roles/instance profiles over hard-coded long-lived AWS credentials on EC2.
- Restrict security-group inbound rules to only the ports required by the implementation.
- Use controlled access to Jenkins rather than exposing administrative interfaces unnecessarily.
- Use versioned ECR image tags for traceability in a production workflow.
- Use HTTPS/TLS and stronger access controls for a production deployment.

13. Advantages

- Automated and repeatable Java application delivery.
- Reduced manual deployment operations.
- Consistent Docker-based runtime environment.
- Centralized CI/CD visibility through Jenkins.
- Private image storage using Amazon ECR.
- Clear separation between source control, CI orchestration, registry and runtime.
- Pipeline logs and stage visualization make implementation progress easier to verify.

14. Limitations

- The Level 5 implementation uses a Docker EC2 host rather than a full container orchestration platform.
- The application is demonstrated on port 8080 and is intended as a learning/project environment.
- A single Docker host does not provide the same high-availability capabilities as a multi-instance orchestrated deployment.
- Advanced deployment strategies and centralized production-grade observability are outside the demonstrated Level 5 scope.

15. Future Enhancements

- Deploy containers to Amazon ECS or Amazon EKS.
- Introduce an Application Load Balancer and HTTPS/TLS for external access.
- Implement rolling or blue/green deployment strategies.
- Add automated source-code and container-image security scanning.
- Add CloudWatch/Prometheus/Grafana monitoring and centralized logging.
- Manage AWS infrastructure with Terraform.
- Use Git commit IDs or release versions as ECR image tags.
- Implement automated rollback when deployment health checks fail.

16. Conclusion

The Level 5 project demonstrates a practical CI/CD implementation for a Java/Tomcat web application. GitHub provides source control and webhook integration, Jenkins orchestrates the pipeline, Maven produces the WAR artifact, Docker packages the application, Amazon ECR stores the container image, and AWS EC2 provides the runtime host. The supplied implementation evidence documents the setup sequence, troubleshooting, pipeline execution, ECR configuration, IAM attachment and final SampleApp validation.

The complete demonstrated flow is:

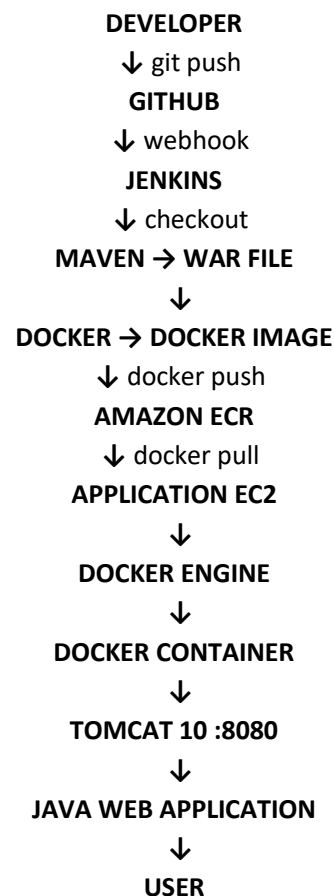
**Developer → GitHub → Jenkins → Maven → WAR → Docker → Amazon ECR → EC2 → Docker → Tomcat 10 → Java
Web Application → User**

17. Implementation Evidence Summary

A total of 25 individual implementation screenshots were supplied for this documentation, together with 1 separate system architecture diagram. The images covers the GitHub repository, Jenkins EC2 environment, Jenkins initialization, Java/Maven and Docker setup, AWS CLI, ECR, Git synchronization and commits, Jenkins Pipeline creation, GitHub webhook, pipeline execution, troubleshooting, deployed SampleApp validation, Jenkins final status, IAM/ECR attachment and ECR repository verification.

Images Type	Count
implementation screenshots	25
System architecture diagram	1
Total project images included	26

Appendix A – Complete CI/CD Flow



END OF LEVEL 5 CONTAINERIZED JAVA/TOMCAT APPLICATION CI/CD PROJECT